

网络部分>>

iptables 的功能作用

访问控制与安全防护：充当主机防火墙，允许或拒绝特定 IP、网段、端口的访问
端口映射与目标地址转换，源地址转换与共享上网。DNAT，SNAT

DDoS 攻击的原理，如何防范

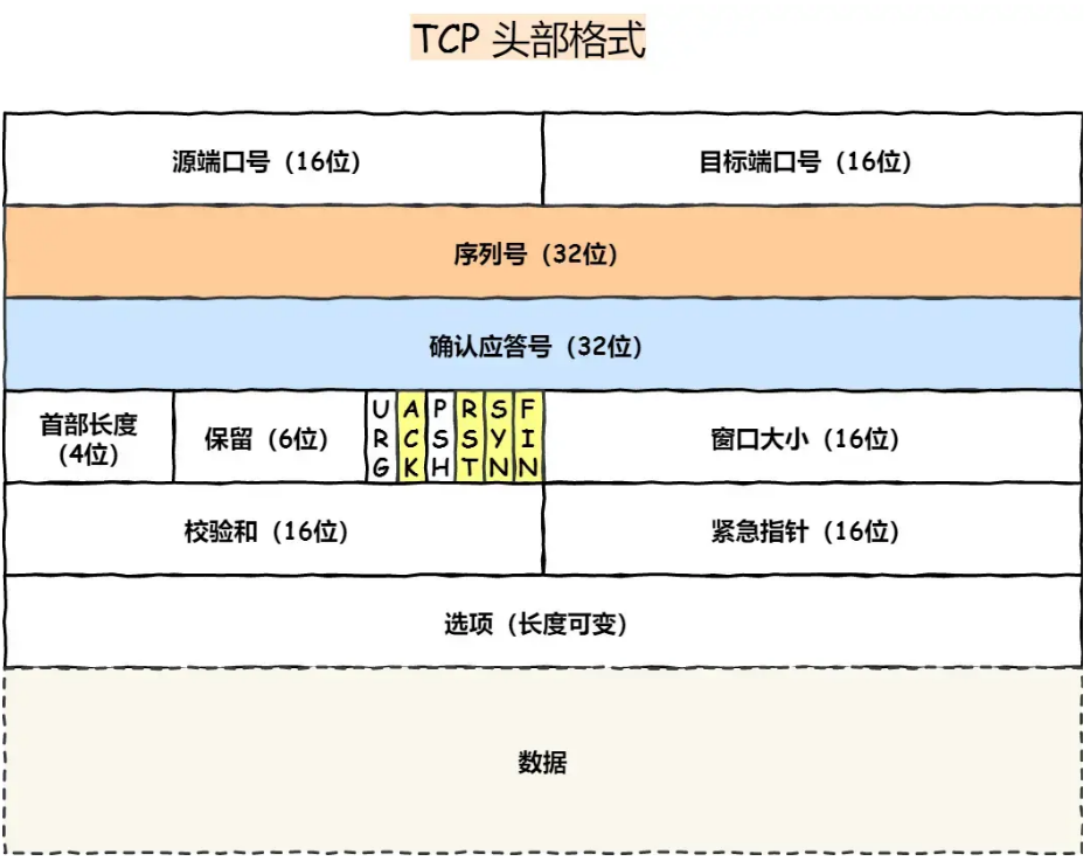
攻击者控制海量僵尸网络设备，向目标服务器发送大量伪造的或合法的请求，耗尽目标的网络带宽、系统连接数或计算资源（CPU/内存），导致正常用户无法访问服务

纵深防御：
通过 DNS 解析将流量牵引到清洗中心，过滤掉恶意报文后，再将干净流量回源到真实服务器
将流量分散到全球多个数据中心，利用分布式网络的整体容量来硬抗攻击

Nginx 限流： 利用前面提到的 limit_req 模块（漏桶算法）限制单一 IP 的请求频率，利用 limit_conn 限制并发连接数

弹性扩容

TCP头部字段格式



HTTP 和 HTTPS 的区别？

- HTTP 是超文本传输协议，信息是明文传输，存在安全风险的问题。HTTPS 则解决 HTTP 不安全的缺陷，在 TCP 和 HTTP 网络层之间加入了 SSL/TLS 安全协议，使得报文能够加密传输。
- HTTP 连接建立相对简单，TCP 三次握手之后便可进行 HTTP 的报文传输。而 HTTPS 在 TCP 三次握手之后，还需进行 SSL/TLS 的握手过程，才可进入加密报文传输。
- 两者的默认端口不一样，HTTP 默认端口号是 80，HTTPS 默认端口号是 443。
- HTTPS 协议需要向 CA（证书权威机构）申请数字证书，来保证服务器的身份是可信的。

HTTPS 的加密流程是怎样的？

1. 客户端向服务器发送请求，包含支持的密码套件列表，随机数1
 2. 服务端收到后，从客户端给出的选项里挑一套双方都支持的算法，并回复选中的密码套件和随机数2
 3. 服务端会把自己的数字证书发给客户端，证书里面包含服务端的公钥和权威机构的数字签名
 4. 客户端拿到证书后，本地校证书，生成随机数3，用服务器公钥加密，发给服务端

说一下GET和POST的区别？

GET 是安全的：它的语义是“获取”资源，不会对服务器上的数据进行修改

POST 是不安全的：它的语义是“创建或修改”资源，会改变服务器的状态

GET 是幂等的：执行一次和执行一万次，对服务器产生的影响是一样的

POST 不是幂等的

Linux内核部分 >>

top的各个参数

字母 / 字段	含义（通俗解释）	高频场景
PID	进程 ID	杀进程 / 查进程用 kill PID
USER	进程所属用户	区分 root / 普通用户进程
PR	进程优先级（内核动态调整）	数值越小优先级越高
NI	进程 nice 值（手动优先级）	范围 -20~19，-20 优先级最高
VIRT	进程占用的虚拟内存总量	包含未实际使用的内存（参考性低）
RES	进程实际占用的物理内存（常驻）	核心关注！= 应用真实内存占用
SHR	进程共享内存大小	比如共享库占用的内存
S	进程状态	R = 运行 / 睡眠，S = 休眠，Z = 僵尸，D = 不可中断睡眠
%CPU	进程占用 CPU 百分比	按 P 可按 CPU 降序排序
%MEM	进程占用物理内存百分比	按 M 可按内存降序排序
TIME+	进程累计占用 CPU 时间	比如 10:05.23 = 10 小时 5 分钟

OOM 情况诊断以及处理措施

1. 先诊断：用top/ps定位高内存进程，判断是泄漏、配置、流量还是资源问题
2. 杀进程、重启服务、释放缓存、保护核心进程
3. 紧急止血，临时扩容，提高核心进程优先级，完善监控体系

服务器高负载情况下会对服务产生什么具体影响

1. 操作系统频繁进行上下文切换，请求处理延迟增加
2. 网关/代理层连接超时，触发504，502错误
3. 大量进程在等待磁盘读写，陷入不可中断睡眠状态，业务无法响应请求
4. 触发 OOM Killer 强杀核心进程，引发雪崩效应

select/poll/epoll

select 是最早的 I/O 多路复用机制，让操作系统去盯着，谁有数据来了就告诉应用程序
数据来了后，调用**select**，把包含所有文件描述符的集合从用户态全量拷贝到内核态，内核拿到后，需要线性遍历，检查是否有事件发生，如果有，标记为“就绪”，然后再把整个集合全量拷贝回用户态，应用程序拿到集合后，还需再次遍历。缺点：监听数量有限，来回拷贝，开销大

poll 换汤不换药，就是改变了结构体，突破了1024的数量限制

epoll_ctl 每次有新的连接接入，只需要调用一次它，把新的 FD 挂载到红黑树上，当网卡收到数据并触发硬中断时，回调函数会自动把这个 FD 放入就绪队列中

epoll_wait 调用时，根本不需要遍历所有的 FD。它只需要检查内核中的就绪队列是否为空

LT水平触发：只要内核的缓冲区里还有数据没读完，每次调用 **epoll_wait**都会返回就绪状态，直到把数据读干净，ET边缘触发，只有在状态发生变化时，才会通知，系统调用次数更少，适合高并发场景

Linux操作系统部分>>

Linux 用的发行版，几个发行版之间的区别

了解不多，一般是用**centos** 和 **ubuntu**
centos ：保守，版本老，以稳定为主，传统企业级后端服务
ubuntu ：相对激进，适合云原生、快速迭代的互联网业务

Linux操作系统结构

/var: 存放系统中经常发生变化的数据
/tmp: 存放系统和程序的临时文件

/etc: 系统的“控制中心”，存放了所有的系统管理和全局配置文件
/boot: 存放系统启动时所需的核心文件

/proc: 极其重要！它包含了当前系统运行的进程信息和内核参数
/dev : 存放所有的硬件设备文件，比如硬盘等等

/opt: 通常用于存放第三方安装的、非系统自带的大型可选软件包

中间件与高可用>>

针对于499问题，有什么解决办法（nginx独有）

如果是后端处理太慢（这是一个耗时的任务），建议重构为：前端请求 -> 后端立刻返回 202 -> 任务扔进消息队异步处理 -> 前端通过轮询获取结果

如果是超时时间设置不合理，需要同步时间。

有些时候，用户等不及刷新了页面（触发 499），但这个请求涉及核心业务逻辑（比如扣款、下单、写入核心数据），我们希望即使客户端断开了，后端的逻辑也要强行执行完，开启这个配置后，Nginx 会忽略客户端的断开事件，继续等待后端处理完毕

云原生部分>>

Docker 子网内通信方式 / 以及实现原理

每个容器启动时，Linux 会为它分配一个独立的 Network Namespace，意味着它有自己独立的网卡、IP 地址、路由表和 iptables 规则

eth0 ----- veth（网桥docker0）

本质上并没有经过复杂的宿主机三层路由转发，而是通过虚拟网桥在二层（数据链路层）直接进行了 MAC 地址转发，性能损耗非常小

核心靠 3 大底层技术 + 一套镜像管理机制实现

Namespace：实现资源隔离，让容器“看起来”像一台独立的 Linux 系统，与其他容器 / 宿主机完全隔离

Cgroups：实现资源限制，限制容器能使用的最大资源，防止单个容器耗尽宿主机资源

UnionFS（联合文件系统）：实现镜像分层，不同镜像可共享底层层，极大节省存储空间

多个 Cgroup 是否可以对同一个进程加以限制

v1：不同子系统的 Cgroup 可以共同限制同一个进程（cpu/group_A, memory/group_B），否则不行

v2：不可以。一个进程只能属于唯一的一个 Cgroup （v1配置太麻烦）

基于 Jenkins Gitlab 实现 CI/CD 流程的描述

1. 研发人员在本地完成 Python 代码开发后，将代码 git push 到 GitLab，GitLab 捕获到 push 事件后向 Jenkins 的 API 发送一个 HTTP POST 请求，通知 Jenkins 开始干活
2. Jenkins 接收到触发信号后，拉取代码，执行自动化测试，基于代码仓库中的 Dockerfile，将 Python 应用及其依赖打包成一个 Docker 镜像，为镜像打上唯一的 Tag，推送到镜像仓库
3. Jenkins 会修改目标环境的 Kubernetes 部署文件，将其中的 image 字段更新

什么是服务网格？

将网络通信相关的逻辑全部从业务代码中剥离出来，下沉到独立的基础设施层

服务网格在逻辑上严格分为两层

数据平面：Sidecar 代理，负责实际流量转发、治理

控制面：统一配置管理（如 Istio），下发规则给所有 Sidecar

数据库部分 >>

MYSQL数据库有哪些常见索引?

按“数据结构”分类

B+ 树索引：非叶子节点只存储索引键值（不存真实数据），所有真实数据行都存在最底层的叶子节点中，叶子节点之间用双向链表连接，非常适合做范围查询

哈希索引：查找速度极快，复杂度低，但是不支持范围查询

按“物理存储”分类

聚簇索引：表里的整行数据，就挂在这个索引树的叶子节点上

二级索引：针对其他列建的索引，叶子节点存的是主键的值，而不是整行数据

按“逻辑应用”分类

联合索引：把多个列组合在一起，建一个索引。

最左前缀匹配原则： 使用联合索引时，查询条件必须从索引的最左列开始，并且不能跳过列，不然会触发全表扫描

说一下事务隔离级别，InnoDB默认是哪种隔离级别?

读未提交：存在脏读，性能高

读已提交：存在不可重复读

可重复读：**MySQL InnoDB 默认级别**，兼顾一致性和性能，适合绝大多数业务